UNIVERSIDAD POLITÉCNICA DE CHIAPAS

Alumnos:
Angel Eduardo Guzmán López

Grado y grupo: 4° B

Asignatura: Estructura de datos

Docente: ELIAS BELTRAN NATURI

U2ACT0301 - EJ PILAS

05.10.2025

Introducción

Dentro de este informe se detalla la resolución para los problemas de Pilas.
Los dos problemas son:

1. Historial de Navegación (Atrás/Adelante)
2. Gestor de Deshacer / Rehacer

La característica de una pila en estos ejercicios es el LIFO (Last-in, First-out / Último en entrar, primero en salir) es clave para modelar el comportamiento de ambos ejercicios.

Historial de Navegación

Planteamiento:

Se debe simular el historial de un navegador web usando 3 Historias de usuarios (HU):

- HU-01: Poder regresar a la página anterior
- HU-02: Después de ir atrás, poder avanzar a la página que se dejó (adelante)
- HU-03: Si se visita una página nueva después de ir "atrás", el historial "adelante" debe borrarse.

Modelo de dos pilas

```
static class HistorialNavegador {
    private Stack<PaginaWeb> pilaAtras;
    private Stack<PaginaWeb> pilaAdelante;
    private PaginaWeb paginaActual;
```

1. pilaAtras (Stack<PaginaWeb>): Se encarga de almacenar las páginas que hemos dejado atrás. La cima de esta pila es la página a la que iremos si presionamos "Atrás".
2. pilaAdelante (Stack<PaginaWeb>): Se encarga de almacenar las páginas que hemos dejado adelante (es decir, solo se llena después de presionar "Atrás"). La cima de esta pila es la página a la que iremos si presionamos "Adelante".
3. paginaActual (PaginaWeb): Una variable simple que almacena la página que el usuario está viendo en este momento.

Implementación de la solución

```java
// nuevaPagina La nueva página a visitar
public void nuevaPagina(PaginaWeb nuevaPagina) {
    System.out.println("Estamos visitando: " + nuevaPagina);

    if (paginaActual ≠ null) {
        pilaAtras.push(paginaActual);
    }

    // la nueva pagina es ahora la actual
    paginaActual = nuevaPagina;

    // Si visitamos una nueva, la pila de adelante se borra.
    if (!pilaAdelante.isEmpty()) {
        System.out.println(" ── Borrando historial 'adelante' ── ");
        pilaAdelante.clear();
    }
}
```

nuevaPagina(PaginaWeb nuevaPagina)
Este método se llama cuando el usuario escribe una URL nueva
Su propósito es establecer una nueva página como la actual, actualizar el historial “Atrás” y borrar el historial de “Adelante”

- Algoritmo
    - Verificar si paginaActual no es null (es decir, si no es la primera pagina que estamos visitando)
    - Si no es null empuja (push) la paginaActual en la pilaAtras
    - Establece la nuevaPagina como la paginaActual
    - Verifica si la pilaAdelante no está vacía.
    - Si no está vacía limpia la pila por completo con pilaAdelante.clear(). Esto es fundamental ya que al visitar un nuevo enlace se crea una nueva rama de navegacion invalidando así el antiguo historial de “Adelante”.

navegarAtras()

```java
public void navegarAtras() {
    if (!pilaAtras.isEmpty()) {
        System.out.println("←── ATRAS: no hay mas páginas tilin");
        return;
    }

    // se guarda la actual en la de adelante
    pilaAdelante.push(paginaActual);

    // sacamos la de atras y se muestra
    paginaActual = pilaAtras.pop();

    System.out.println("←── Atras: cargando " + paginaActual);
}
```

Este método se llama cuando el usuario presiona el botón “Atrás”.
Su propósito es mover la paginaActual a la pila “Adelante” y cargar la página de la cima de la pila “Atrás”

- Algoritmo
    - Verificar si la pilaAtras esta vacia con pilaAtras.isEmpty().
    - Si está vacía no se realiza ninguna acción
    - Si no está vacía se empuja (push) la paginaActual en la pilaAdelante. Sacar (pop) la página de encima de pilaAtras y asignarla como la nueva paginaActual.

navegarAdelante()

```java
public void navegarAdelante() {
    if (pilaAdelante.isEmpty()){
        System.out.println("⟶ Adelante: No hay más paginas tilin");
        return;
    }

    // Se guarda la pagina actual en la pila Atras
    pilaAtras.push(paginaActual);

    // Se muestra la pila de adelante
    paginaActual = pilaAdelante.pop();

    System.out.println("⟶ Adelante: cargando " + paginaActual);
}
```

Este método se llama cuando el usuario presiona el botón “Adelante”.
Su propósito es mover la paginaActual a la pila de atrás y cargar la página de la cima de la pila adelante.

- Algoritmo:
    - Verifica si pilaAdelante está vacía con pilaAdelante.isEmpty()
    - Si está vacía no realiza ninguna acción
    - Si no está vacía: empuja la paginaActual en la pilaAtras. Saca la página de la cima de pilaAdelante y asignarla como la nueva paginaActual.

Gestor de Deshacer

planteamiento:

Se debe simular la funcionalidad de Ctrl+z (Deshacer) y Ctrl+y (Rehacer) de un editor de texto.

- HU-04: Poder revertir la última acción (Deshacer)
- HU-05: Tras deshacer, poder recuperar la acción (Rehacer).
- HU-06: Si se realiza una acción nueva, la pila “Rehacer” debe limpiarse.

Solución:

1. IComando (Interface): una interfaz que define dos métodos clave:
    a. ejecutar(): Realiza la acción
    b. deshacer(): Revierte la acción

```java
interface IComando {
    void ejecutar();
    void deshacer();
}
```

2. Comandos Concretos (Clases). clases que implementan IComando.
    a. ComandoInsertar(): Su ejecutar() inserta un texto. Su deshacer() borra ese mismo texto

```java
class ComandoInsertar implements IComando {
    private DocumentoTexto documento;
    private String textoAInsertar;
    private int posicion;

    public ComandoInsertar(DocumentoTexto doc, String texto, int pos) {
        this.documento = doc;
        this.textoAInsertar = texto;
        this.posicion = pos;
    }

    @Override
    public void ejecutar() {
        // Ejecutar es insertar el texto
        documento.insertar(posicion, textoAInsertar);
    }

    @Override
    public void deshacer() {
        // Deshacer una inserción es borrar ese mismo texto
        documento.borrar(posicion, textoAInsertar.length());
    }
}
```

b. ComandoBorrar(): Su ejecutar() borra texto y debe guardar su texto borrado. Su deshacer() inserta el texto que se guardó.

```java
class ComandoBorrar implements IComando {
    private DocumentoTexto documento;
    private String textoBorrado;
    private int posicion;
    private int longitud;

    public ComandoBorrar(DocumentoTexto doc, int pos, int len) {
        this.documento = doc;
        this.posicion = pos;
        this.longitud = len;
    }

    @Override
    public void ejecutar() {
        // Antes de borrar, guarda el texto que va a ser eliminado
        this.textoBorrado = documento.texto.substring(posicion, posicion + longitud);
        // Ejecutar es borrar el texto
        documento.borrar(posicion, longitud);
    }

    @Override
    public void deshacer() {
        // Deshacer un borrado es insertar el texto que guardamos
        documento.insertar(posicion, textoBorrado);
    }
}
```

3. Las pilas: Al igual que en el primer ejercicio usamos pilas pero esta vez almacenan IComando, no páginas
    a. pilaDeshacer(Stack<IComando>): Se encarga de almacenar las acciones que se han ejecutado.
    b. pilaRehacer(Stack<IComando>): Se encarga de almacenar las acciones que se han deshecho.

Implementación:

ejecutarAccion(IComando comando)

Este método se llama cuando el usuario realiza una nueva acción
Su propósito es ejecutar una nueva acción, guardada para “Deshacer” y limpiar la pila “Rehacer”

- Algoritmo:
    - Llamar al método comando.ejecutar() esto modifica el documento real.
    - Empuja (push) ese comando en la pilaDeshacer
    - Verifica si la pilaRehacer no está vacía.
    - Si no está vacía limpia la pila por completo con pilaRehacer.clear()

```java
public void ejecutarAccion(IComando comando) {
    System.out.println("Ejecutando acción ... ");
    comando.ejecutar();

    // La acción se apila en "Deshacer"
    pilaDeshacer.push(comando);


    if (!pilaRehacer.isEmpty()) {
        System.out.println("   (Borrando historial 'rehacer' ... )");
        pilaRehacer.clear();
    }
}
```

deshacer()

Este método se llama cuando el usuario presiona Ctrl+Z
Su propósito es revertir la última acción y moverla a la pila “Rehacer”

- Algoritmo:
    - Verifica si la pilaDeshacer esta vacia con pilaDeshacer.isEmpty()
    - Si está vacía no realiza ninguna acción
    - Si no está vacía saca (pop) el comando de la cima de pilaDeshacer. Llama al método comando.deshacer(). Esto revierte el cambio en el documento. Empuja ese mismo comando en la pilaRehacer

```java
public void deshacer() {
    if (pilaDeshacer.isEmpty()) {
        System.out.println("DESHACER: Nada que deshacer.");
        return;
    }


    IComando comando = pilaDeshacer.pop();

    comando.deshacer();
    System.out.println("DESHACER: Acción revertida.");

    pilaRehacer.push(comando);
}
```

rehacer()

Este método se llama cuando el usuario presiona Ctrl+Y
Su propósito es re-ejecutar la última acción deshecha y moverla de nuevo a la pila “Deshacer”

- Algoritmo:
    - Verifica si pilaRehacer esta vacia con pilaRehacer.isEmpty
    - Si está vacía no realiza ninguna acción
    - Si no está vacía saca el comando de la cima de pilaRehacer. llama al método comando.ejecutar(). esto se re-aplica el cambio en el documento. empuja ese mismo comando de nuevo en la pilaDeshacer

```java
public void rehacer() {
    if (pilaRehacer.isEmpty()) {
        System.out.println("REHACER: Nada que rehacer.");
        return;
    }

    IComando comando = pilaRehacer.pop();

    comando.ejecutar();
    System.out.println("REHACER: Acción restaurada.");

    pilaDeshacer.push(comando);
}
```